HAZMAT SAFETY DATA SHEET

0x3212b29E33587A00FB1C83346f5dBFA69A458923 (verified as IMBTC)

Read 22,174 bytes of deployed bytecode and checked it against 16 registered hazards. 1 hazard is present, reproduced against this very contract.

What this is: It is a plain contract on mainnet, read from its bytecode.
What matched: 1 of the 16 registered hazards is present: HZ-016 (asset-handling).
How sure: HZ-016 was reproduced against this exact contract, not a lookalike.
The sharpest one: HZ-016: A token that calls the recipient back mid-transfer drains a counterparty that settles after paying
What is at stake: up to the credited balance a naive counterparty is holding; about 25M USD in one event against imBTC in 2020.
Precedent: Lendf.Me, 2020-04-19, roughly 25M USD, re-entrancy through imBTC's ERC777 callback.
What to do about it: Not a token bug: any contract that accepts this token must clear its credit before paying out, or re-enter-proof its payout path. Where the counterparty cannot be changed, deposit through a wrapper that uses transfer() and keeps the callback off the hot path.

ASSERTION LEVEL: PATTERN MATCH ONLY; CONFIRMATION RUNS ARE SEPARATE
A match is a candidate, not a clearance: this sheet fingerprints a contract against registered hazard shapes, it does not audit it. A clean sheet is not the same as safe.

ANSWERS TO THE USUAL QUESTIONS
read from the selectors this contract exposes, not from its docs. Every answer says what it cannot tell you.

Did you find any vulnerabilities in the contract?
No audit happened, so "found" is the wrong word. But HZ-016 is not a shape: the behaviour was reproduced against this exact contract, not a lookalike - a token that calls the recipient back mid-transfer drains a counterparty that settles after paying. That makes it a demonstrated hazard rather than a candidate; it becomes a finding when somebody shows it is reachable, and a bug when the code is not meant to behave that way. Nothing else matched.
seen: no matching selector, so this is its shape
not answered here: Whether anything here is reachable, and whether it is a bug rather than a decision. Presence is not proof; absence is not clearance.

So is it safe to use?
Unknown, and this sheet will not pretend otherwise. What it can say is narrow and true: which hazard shapes are present (1), which behaviours have been demonstrated (a placard that says confirmed), and which classes the code does not match at all. What it cannot say is whether any of them is reachable, whether the economics make it exploitable, or whether a hazard is a bug or a deliberate choice - intent is not in the bytecode.
seen: no matching selector, so this is its shape
not answered here: The honest one: the state of the owner key, what you have approved, and what happens to the contract after this block.

What is this?
It is a token contract verified as IMBTC. Everything below is read from the bytecode it exposes, not from its docs.
seen: no matching selector, so this is its shape
not answered here: Whether it is any good, and who wrote it.

Can they freeze or block my tokens?
Yes, transfers can be paused: the contract carries pause/unpause, so a holder of that key can stop everyone moving tokens, including you.
seen: 0x8456cb59 pause(), 0x3f4ba83a unpause(), 0x5c975abb paused()
not answered here: Who holds the pause key, and whether the pause affects withdrawals too.
related hazard: HZ-013

Can transfers be switched off entirely?
Yes. Transfers can be turned on and off as a switch, which is how some tokens spend their first days restricted to a whitelist.

seen: 0xb187984f disableTransfer(), 0xf1b50c1d enableTransfer(), 0x92ff0d31 transferable()
not answered here: Whether it is currently on, and who can flip it.

Is the supply fixed?
No. The supply can be minted, so what you hold is a share of something that can grow. Minting is not inherently bad; unminted-and-unbounded is what dilutes you.
seen: 0xdcdc7dd0 mint(address,uint256,bytes,bytes), 0xaa271e1a isMinter(address), 0x983b2d56 addMinter(address)
not answered here: Whether there is a cap, and who holds the mint key.

Who is in charge?
A single address holds admin rights, and the handover takes two steps, which is the safer version, and that address is not visible from bytecode at all.
seen: 0x8da5cb5b owner(), 0xf2fde38b transferOwnership(address), 0x8f32d59b isOwner()
not answered here: Whether that address is an EOA, a multisig or a timelock. Check it before trusting the admin.

Can whoever runs it take the money?
Partly, and on purpose: there are admin paths to move value out (a sweep, a rescue, or a revenue redirect). They usually cannot touch your balance directly, but they can change where the money goes.
seen: 0x2d07953a distributeRevenue(), 0x45338d63 setRevenueAddress(address)
not answered here: Whether those paths can reach your funds or only the contract's own leftovers.
related hazard: HZ-005

Can anyone else move my balance?
Only with your approval: this is an allowance-based token, so the standing risk is what you have approved, not what the contract can do on its own.
seen: 0x095ea7b3 approve(address,uint256), 0x23b872dd transferFrom(address,address,uint256), 0xdd62ed3e allowance(address,address)
not answered here: What you have approved right now - that is state, not bytecode.

What happens when a transfer lands?
The recipient is called back mid-transfer, before the transfer returns. That is what makes a counterparty that credits first and settles after paying re-enterable - the shape of the Lendf.Me drain.
seen: 0x556f0dc7 granularity(), 0x06e48538 defaultOperators(), 0x9bd9bbc6 send(address,uint256,bytes)
not answered here: Whether any particular pool or vault you use is written safely. That is its own scan.
related hazard: HZ-016

BOTTOM LINE

1 of 16 registered hazards are present (HZ-016). That is the fingerprint, not a verdict: somebody still has to read the code, and the sharpest one has already been reproduced against this very contract, which raises it from a shape to a demonstrated behaviour.

BEFORE YOU TRUST IT, CHECK

- Not a token bug: any contract that accepts this token must clear its credit before paying out, or
- re-enter-proof its payout path. Where the counterparty cannot be changed, deposit through a
- wrapper that uses transfer() and keeps the callback off the hot path.
- Find out what the owner address actually is: an EOA, a multisig or a timelock.

HAZARDS FOUND

HZ-016 A token that calls the recipient back mid-transfer drains a counterparty that settles after paying
class 6 asset-handling  CONFIRMED ON THIS TARGET
This token is an ERC777: transferring to a contract calls that contract back before the transfer returns, resolved through the ERC1820 registry, and the ERC777 send() additionally requires a contract recipient to have registered an implementer. A counterparty that credits on deposit and only clears the credit after paying out can therefore be re-entered by the recipient, paid twice out of one balance, and left unable to pay the next honest depositor. The token is not at fault and is behaving to spec. imBTC was the token in the Lendf.Me drain of 2020-04-19, roughly 25 million dollars in one

re-entrancy against exactly this shape, which is why a token whose selector set says ERC777 is worth knowing about before it is accepted, not after.
What is at stake: up to the credited balance a naive counterparty is holding; about 25M USD in one event against imBTC in 2020.
What to do: Not a token bug: any contract that accepts this token must clear its credit before paying out, or re-enter-proof its payout path. Where the counterparty cannot be changed, deposit through a wrapper that uses transfer() and keeps the callback off the hot path.
Reproduced here: confirmations/HZ-016-3212b29e.json - a rebuild of the verified source and the probes in it passed. Confirming behaviour, not intent.
Lendf.Me, 2020-04-19, roughly 25M USD, re-entrancy through imBTC's ERC777 callback.
matched by selectors: 0x9bd9bbc6 0x62ad1b83 0x06e48538 0x556f0dc7 0x959b8c3f 0xd95b6371

WHAT WAS READ
a match means the selectors a placard asks about are present. It does not mean reachable, and a clean sheet is not a clearance.
placards checked 16 classes 7 selectors in code 42 the register asks about 12 otherwise unaccounted 30
0x06fdde03 name()
0x095ea7b3 approve(address,uint256)
0x23b872dd transferFrom(address,address,uint256)
0x2d07953a distributeRevenue()
0x3092afd5 removeMinter(address)
0x313ce567 decimals()
0x3ba0b9a9 exchangeRate()
0x3e9491a2 revenue()
0x45338d63 setRevenueAddress(address)
0x5c975abb paused()
0x75ab9782 tokensToSend(address,address,address,uint256,bytes,bytes)
0x8f32d59b isOwner()
0x92ff0d31 transferable()
0x95d89b41 symbol()
0x983b2d56 addMinter(address)
0xa9059cbb transfer(address,uint256)
0xaa271e1a isMinter(address)
0xaabbb8ca getInterfaceImplementer(address,bytes32)
0xb187984f disableTransfer()
0xb410908d revenueAddress()
... and 4 more

NOT CHECKED AT ALL
- resolution is best effort: unknown proxy patterns, diamond facets and beacons pointing at beacons
- are labelled not followed
- reachability
- economic preconditions
- state at any particular block
- source-level reasoning
- anything not covered by a placard

THE TECHNICAL PART
target 0x3212b29E33587A00FB1C83346f5dBFA69A458923
family evm
code sha256 0xddb50063c72473b19188a335c6c7a6072b2899657c980147d37045fccb9c0603
body bytes read 22174
metadata excluded 43 bytes (CBOR trailer, appended after compilation)
four-byte constants 42
register 16 placards v0xd7dea67cdf60b0bc
anchor epoch 3 root 0xed6a388e080b234215fcd1d0fc845207714daece4261c0ff9d7e39ce45c5a26c (epoch-3.json)
register digest 0xd7dea67cdf60b0bc
anchor root 0xed6a388e080b234215fcd1d0fc845207714daece4261c0ff9d7e39ce45c5a26c
selector index 783 names, a convenience reading and not verification

Cite the placard id, the code sha256 and the register digest. The HTML sheet is the artifact that carries its own digest; this PDF is the sam